

FLEETMANAGER MANUTENZIONE

1. INTRODUZIONE

1.1 Scopo del documento

Lo scopo del presente documento è descrivere le attività di manutenzione svolte per il progetto FleetManager, sviluppato nell'ambito del corso di Ingegneria del Software A.A. 2025/26. Il documento illustra come il software sia stato corretto, migliorato ed evoluto durante il ciclo di sviluppo, evidenziando le principali modifiche apportate al codice e le motivazioni tecniche che le hanno guidate.

Nel contesto di FleetManager, la manutenzione non è intesa come attività post-rilascio in un ambiente produttivo, ma come insieme di interventi correttivi ed evolutivi svolti in modo continuativo durante lo sviluppo iterativo-incrementale, con l'obiettivo di mantenere il sistema stabile, coerente con i requisiti e facilmente estendibile.

1.2 Ambito della manutenzione

Le attività di manutenzione descritte in questo documento riguardano la versione attuale del sistema FleetManager e si concentrano principalmente su:

- **manutenzione correttiva**, legata alla risoluzione di anomalie e casi limite emersi durante lo sviluppo e l'esecuzione dei test;
- **manutenzione evolutiva**, relativa al raffinamento progressivo delle funzionalità e delle regole di business;
- **manutenzione preventiva**, con interventi di refactoring finalizzati a migliorare manutenibilità, leggibilità del codice e qualità complessiva del progetto.

Il documento non descrive attività di manutenzione svolte dopo un rilascio ufficiale a utenti reali, né include procedure operative di deployment o assistenza in esercizio, poiché FleetManager è un progetto accademico e viene eseguito in ambiente locale.

1.3 Riferimenti

Le attività di manutenzione sono state svolte facendo riferimento a:

- le **linee guida del corso di Ingegneria del Software** (UniBG A.A. 2025/26);
- i principi descritti in *Software Engineering: Principles and Practice* di **Van Vliet**, con particolare attenzione ai concetti di manutenibilità, refactoring e gestione della qualità;

- il documento di **Analisi dei Requisiti**, utilizzato come baseline per verificare coerenza tra modifiche e requisiti;
- il documento di **Testing**, utilizzato come riferimento per la validazione delle modifiche e per l'identificazione di regressioni tramite test automatici JUnit.

2. TIPOLOGIE DI MANUTENZIONE

2.1 Manutenzione correttiva

Nel progetto FleetManager sono state svolte diverse attività di manutenzione correttiva durante lo sviluppo, principalmente in risposta a comportamenti non corretti emersi dall'esecuzione dei test automatici e dall'utilizzo dell'interfaccia grafica.

Tali interventi hanno riguardato, ad esempio, la correzione della logica di gestione delle prenotazioni, l'adeguamento dei controller della GUI e il perfezionamento dei casi di test relativi ai DAO e al Service Layer. In più occasioni, l'esecuzione dei test JUnit ha permesso di individuare incongruenze o regressioni introdotte durante l'evoluzione del codice, che sono state risolte mediante interventi mirati di correzione. La manutenzione correttiva è stata quindi strettamente integrata con l'attività di testing, contribuendo a mantenere il sistema stabile e coerente con i requisiti funzionali definiti.

2.2 Manutenzione evolutiva

La manutenzione evolutiva ha rappresentato una parte significativa dello sviluppo di FleetManager. Il sistema è stato progressivamente arricchito e raffinato attraverso l'introduzione e il miglioramento di servizi applicativi dedicati alla gestione delle principali funzionalità di business.

In particolare, sono stati introdotti e successivamente perfezionati componenti come il gestore delle prenotazioni, delle scadenze, delle manutenzioni e del sistema di notifiche. Alcune funzionalità sono state inizialmente implementate in forma semplificata e successivamente estese, ad esempio con l'introduzione di metodi specifici per la gestione dello stato delle prenotazioni e con l'automatizzazione di operazioni che inizialmente venivano gestite manualmente. Questi interventi riflettono un approccio iterativo–incrementale, in cui il software viene continuamente adattato e migliorato in base a una comprensione più approfondita dei requisiti e del dominio applicativo.

2.3 Manutenzione adattiva

Nel corso del progetto sono state svolte anche attività riconducibili alla manutenzione adattiva, finalizzate ad adeguare il sistema al contesto di sviluppo e di esecuzione. Esempi significativi includono l'adozione di un database embedded H2, l'introduzione di componenti di supporto per la gestione automatica del database durante i test e l'adattamento del codice per supportare l'integrazione con l'interfaccia grafica JavaFX. Questi interventi hanno permesso al sistema di operare correttamente in un ambiente di sviluppo locale, facilitando sia il testing automatico sia l'evoluzione dell'applicazione.

2.4 Manutenzione preventiva

Sono state svolte diverse attività di manutenzione preventiva con l'obiettivo di migliorare la qualità interna del sistema e ridurre il rischio di problemi futuri.

In particolare, il codice è stato oggetto di refactoring strutturali, come la riorganizzazione dei package, la separazione più chiara tra interfacce e implementazioni dei DAO, e la revisione complessiva di parti del codice per migliorarne leggibilità e manutenibilità. L'introduzione di classi di utilità, come quelle dedicate al supporto dei test, ha contribuito a rendere il sistema più robusto e facilmente modificabile. Questi interventi preventivi hanno avuto un ruolo fondamentale nel supportare l'evoluzione del progetto e nel mantenere il codice comprensibile e gestibile nel tempo.

3. ATTIVITÀ DI REFACTORING SVOLTE

3.1 Motivazioni del refactoring

Durante lo sviluppo di FleetManager sono state svolte diverse attività di refactoring con l'obiettivo di migliorare la qualità interna del sistema, facilitarne la manutenzione e supportarne l'evoluzione nel tempo. Le principali motivazioni che hanno portato all'introduzione di interventi di refactoring sono state:

- la necessità di migliorare la **leggibilità del codice**, man mano che il numero di classi e funzionalità aumentava;
- l'esigenza di ridurre **accoppiamenti non necessari** tra i diversi livelli dell'architettura;
- il supporto allo **sviluppo incrementale**, evitando che modifiche successive rendessero il codice difficile da estendere o testare;
- il miglioramento dell'**affidabilità dei test automatici**, rendendo più semplice il ripristino dello stato iniziale del sistema durante l'esecuzione dei test.

Il refactoring è stato quindi considerato come parte integrante del processo di sviluppo, e non come un'attività separata o finale.

3.2 Riorganizzazione dei package

Nel corso dello sviluppo di FleetManager è stata effettuata una riorganizzazione progressiva dei package, finalizzata a rendere più chiara la struttura del sistema e a migliorare la separazione delle responsabilità tra i diversi componenti. La struttura finale del progetto riflette un'architettura a livelli ben definita, articolata nei seguenti package principali:

- **model**: contenente le classi di dominio che rappresentano le entità fondamentali del sistema (ad esempio veicoli, utenti, prenotazioni, manutenzioni, notifiche e scadenze);
- **repository**: suddiviso ulteriormente in:
 - **dao**: contenente le interfacce per l'accesso ai dati;
 - **impl**: contenente le classi di implementazione dei DAO;
 - **util**: con classi di supporto all'accesso ai dati;
- **service**, organizzato in:

- **interfaces:** che definiscono i contratti dei servizi applicativi;
- **impl:** che forniscono le implementazioni concrete della logica di business;
- **ui:** dedicato alla gestione dell'interfaccia grafica JavaFX, ulteriormente suddiviso in sotto-package funzionali (dashboard, prenotazioni, manutenzioni, notifiche, scadenze, veicoli);
- **util:** contenente classi di supporto condivise, come enumerazioni e utilità comuni.

3.3 Separazione tra modello, accesso ai dati e logica applicativa

Un'importante attività di refactoring ha riguardato la separazione più netta tra il modello di dominio, il livello di accesso ai dati e la logica applicativa.

Le classi presenti nel package *model* rappresentano esclusivamente lo stato e le caratteristiche delle entità del dominio, senza includere logica applicativa o dipendenze verso altri livelli. L'accesso ai dati è stato incapsulato all'interno del package *repository*, attraverso l'uso di interfacce DAO e delle rispettive implementazioni, riducendo l'accoppiamento tra il database e il resto del sistema.

La logica di business è stata concentrata nel package *service*, dove le classi di servizio orchestrano le operazioni sui dati, applicano le regole di dominio e coordinano l'interazione tra *repository* e interfaccia utente. Questo refactoring ha permesso di evitare la dispersione della logica applicativa all'interno dei controller della GUI o delle classi di accesso ai dati.

3.4 Refactoring del livello di accesso ai dati (DAO)

Le classi di accesso ai dati sono state oggetto di ulteriori interventi di refactoring, volti a migliorarne la manutenibilità e la coerenza interna. In particolare, è stata introdotta una distinzione esplicita tra:

- **interfacce DAO:** definite nel package *repository.dao*, che specificano le operazioni disponibili;
- **classi di implementazione:** collocate nel package *repository.impl*, che forniscono la logica concreta di accesso al database.

Questa separazione ha reso il codice più flessibile e più facilmente estendibile, consentendo eventuali modifiche future al meccanismo di persistenza senza impatti diretti sui livelli superiori del sistema. Inoltre, il refactoring dei metodi CRUD ha migliorato la leggibilità del codice e facilitato la scrittura di test automatici mirati.

3.5 Introduzione di classi di supporto e utilità comuni

Nel corso della manutenzione preventiva sono state introdotte e consolidate alcune classi di supporto nel package *util*, con l'obiettivo di ridurre duplicazioni e migliorare la chiarezza del codice. Tra queste rientrano:

- classi di utilità per il testing, come *DatabaseTestUtils*, utilizzate per inizializzare e ripristinare automaticamente lo stato del database durante l'esecuzione dei test;
- enumerazioni dedicate alla rappresentazione di ruoli, stati e tipologie del dominio (ad esempio ruoli utente, stati delle prenotazioni, tipi di manutenzione o notifica).

Queste classi hanno contribuito a rendere il codice più espressivo, riducendo l'uso di valori letterali e migliorando la coerenza complessiva del sistema.

3.6 Benefici del refactoring

Le attività di refactoring svolte hanno prodotto benefici concreti per il progetto FleetManager, tra cui:

- una struttura del codice più **modulare e leggibile**;
- una migliore **separazione delle responsabilità** tra i livelli dell'architettura;
- una maggiore **facilità di manutenzione ed estensione** del sistema;
- un miglior supporto alle attività di testing automatico.

Grazie a questi interventi, il sistema risulta più robusto e maggiormente predisposto a evoluzioni future, in linea con gli obiettivi di qualità e manutenibilità definiti per il progetto.

4. SUPPORTO DELLA MANUTENZIONE TRAMITE TESTING

4.1 Ruolo del testing nelle attività di manutenzione

Nel progetto FleetManager, il testing automatico ha svolto un ruolo centrale nel supportare le attività di manutenzione del software. I test JUnit non sono stati utilizzati esclusivamente per verificare il corretto funzionamento delle funzionalità implementate, ma anche come strumento di supporto alle modifiche e ai refactoring effettuati nel corso dello sviluppo.

La presenza di una suite di test automatizzati ha consentito di individuare tempestivamente comportamenti non corretti introdotti da modifiche al codice e di verificare che le funzionalità esistenti continuassero a rispettare i requisiti definiti. In questo modo, il testing ha contribuito a ridurre il rischio di regressioni e a mantenere il sistema stabile durante le attività di manutenzione correttiva ed evolutiva.

4.2 Test automatici come strumento di regressione

I test JUnit sono stati utilizzati come principale meccanismo di controllo delle regressioni introdotte durante l'evoluzione del sistema. Ogni intervento di modifica o refactoring del codice è stato accompagnato dall'esecuzione dei test esistenti, permettendo di verificare che il comportamento del sistema rimanesse coerente con quello atteso.

In particolare, i test relativi al livello di accesso ai dati (DAO) e al Service Layer hanno permesso di validare le modifiche apportate alle classi di gestione delle prenotazioni, delle manutenzioni, delle scadenze e del sistema di notifiche. Eventuali errori o incongruenze emerse durante l'esecuzione dei test sono stati corretti prima di procedere con ulteriori estensioni del sistema.

4.3 Supporto al refactoring tramite testing

Il testing automatico ha rappresentato un elemento fondamentale anche nel supportare le attività di refactoring. La possibilità di rieseguire rapidamente l'intera suite di test ha consentito di intervenire sulla struttura del codice, come nella riorganizzazione dei package o nella separazione tra interfacce e implementazioni, con un elevato grado di confidenza.

In questo contesto, i test hanno svolto il ruolo di rete di sicurezza, permettendo di verificare che i refactoring strutturali non alterassero il comportamento osservabile del sistema. Ciò ha reso possibile migliorare la qualità interna del codice senza compromettere la correttezza funzionale dell'applicazione.

4.4 Automazione del testing e gestione dello stato

Per garantire l'affidabilità dei test e supportare efficacemente la manutenzione, è stata introdotta una gestione automatica dello stato del database durante l'esecuzione dei test. Attraverso classi di utilità dedicate, il database embedded viene inizializzato e ripristinato in modo uniforme prima o durante l'esecuzione dei casi di test.

Questa automazione ha permesso di rendere i test indipendenti tra loro, evitando effetti collaterali e rendendo ripetibile l'esecuzione della suite di test. Di conseguenza, le attività di manutenzione e modifica del codice possono essere verificate in modo sistematico, riducendo il rischio di introdurre errori difficili da

4.5 Considerazioni sulla copertura dei test

Sebbene non siano state utilizzate metriche automatiche di copertura del codice, la suite di test è stata progettata per coprire le principali funzionalità del sistema e le componenti più critiche dal punto di vista della logica applicativa.

La presenza di test dedicati ai livelli di accesso ai dati e ai servizi principali ha fornito un livello di confidenza adeguato nel supportare le attività di manutenzione e refactoring, risultando coerente con la dimensione e gli obiettivi del progetto.

4.6 Benefici del testing per la manutenzione

L'integrazione del testing automatico nel processo di sviluppo e manutenzione ha prodotto diversi benefici, tra cui:

- maggiore sicurezza durante le modifiche al codice;
- individuazione tempestiva di errori e regressioni;
- supporto efficace alle attività di refactoring;
- miglioramento complessivo della qualità del software.

Il testing si è quindi rivelato uno strumento essenziale per garantire la manutenibilità del sistema FleetManager e per supportarne l'evoluzione nel tempo.

5. MANUTENIBILITÀ DEL SISTEMA

5.1 Scelte progettuali a supporto della manutenibilità

Il sistema FleetManager è stato progettato ponendo particolare attenzione alla manutenibilità del software, intesa come facilità di comprensione, modifica ed estensione del codice nel tempo.

Le principali scelte progettuali che supportano la manutenibilità del sistema includono:

- l'adozione di un'**architettura a livelli**, con una chiara separazione tra modello di dominio, accesso ai dati, logica applicativa e interfaccia utente;
- l'utilizzo di **interfacce** per definire i contratti dei componenti principali (DAO e servizi), riducendo l'accoppiamento tra i diversi livelli;
- l'organizzazione coerente dei package, che rende immediata l'individuazione delle responsabilità delle singole classi;
- l'uso di un **database embedded**, che semplifica l'ambiente di esecuzione e facilita il testing e la manutenzione.

Queste scelte consentono di isolare le modifiche, limitando l'impatto delle variazioni su parti non coinvolte direttamente.

5.2 Manutenibilità e qualità del codice

La manutenibilità del sistema è ulteriormente supportata dalla qualità interna del codice, migliorata attraverso attività di refactoring e revisione continua. L'introduzione di enumerazioni per la gestione di stati, ruoli e tipologie del dominio ha reso il codice più espressivo e meno soggetto a errori dovuti all'uso di valori letterali.

La presenza di una suite di test automatici contribuisce in modo significativo alla manutenibilità, permettendo di verificare rapidamente la correttezza del sistema dopo ogni modifica. Questo

aspetto risulta particolarmente importante in un contesto iterativo–incrementale, in cui il software evolve progressivamente.

Nel complesso, il sistema risulta facilmente comprensibile e modificabile anche da sviluppatori che non hanno partecipato direttamente alle fasi iniziali del progetto.

6. CONCLUSIONI

Il presente documento ha descritto le principali attività di manutenzione svolte durante lo sviluppo del progetto FleetManager, evidenziando come la manutenzione sia stata parte integrante del ciclo di vita del software e non un'attività marginale o successiva.

Attraverso interventi di manutenzione correttiva, evolutiva, adattiva e preventiva, il sistema è stato progressivamente migliorato sia dal punto di vista funzionale sia dal punto di vista della qualità interna. Le attività di refactoring, unite all'uso sistematico del testing automatico, hanno consentito di mantenere il codice stabile, coerente con i requisiti e facilmente estendibile.

Le scelte architetture adottate e l'organizzazione del progetto favoriscono la manutenibilità del sistema e permettono di affrontare con sicurezza eventuali evoluzioni future. Nel complesso, l'approccio seguito risulta coerente con i principi dell'Ingegneria del Software e con gli obiettivi formativi del corso.